Python Engineering - Programming Theory Guide

CLASS 14 - File I/O | Persistencia, representacion y flujo de datos

EX22 introduce una idea mas profunda que `open()`: la diferencia entre **estado en memoria** y **representacion persistente**. Esta guia desarrolla el modelo mental completo.

1. Memoria RAM vs almacenamiento persistente

Cuando Python crea variables, listas, arrays u objetos, normalmente viven en la memoria RAM del proceso.

```
earth_state = VerticalState(...)
trajectory = np.array(...)
```

Mientras el programa corre, esos datos existen y pueden modificarse. Cuando el proceso termina, esa memoria deja de pertenecer al programa.

> Un archivo en disco es diferente: existe fuera del proceso y puede volver a abrirse mañana, desde otra ejecucion o incluso desde otro programa.

RAM	Archivo
Temporal para el proceso	Persistente
Objetos Python ricos	Bytes organizados segun un formato
Muy rapida	Mas lenta que RAM
Se pierde al terminar el proceso	Permanece hasta borrarlo/modificarlo

2. File I/O

I/O significa Input/Output. En archivos:

```
INPUT  -> file -> Python
OUTPUT -> Python -> file
```

> Guardar una trayectoria es Output. Recuperarla desde CSV es Input.

3. El archivo no conoce nuestros objetos Python

Un archivo de texto no guarda directamente un `VerticalState` ni un `np.ndarray`. Guarda bytes que interpretamos como caracteres.

```
Python float 100.0
       |
       | formatting / string conversion
       v
text '100.000000'
       |
       | UTF-8 encoding
       v
bytes on disk
```

> Al leer hacemos el camino inverso.

4. Serializacion y parsing

Serializar significa convertir datos en una representacion que puede almacenarse o transmitirse. En EX22 nuestra serializacion es muy simple: convertimos numeros en texto CSV.

Parsear significa interpretar esa representacion y reconstruir valores con significado.

```
SERIALIZE
[100.0, 20.0, 0.0]
        -> '100.000000,20.000000,0.000000\n'

PARSE
'100.000000,20.000000,0.000000\n'
        -> ['100.000000','20.000000','0.000000']
        -> [100.0,20.0,0.0]
```

Esta idea aparecera tambien con JSON, configuraciones, redes, bases de datos y formatos cientificos.

5. Un file object es un manejador de recurso

El objeto devuelto por `open()` no es el archivo fisico en si. Es una interfaz Python hacia un recurso gestionado por el sistema operativo.

```
file = open(path, 'r')

# file can:
# - read
# - track current position
# - expose metadata/state
# - be closed
```

Por eso es importante cerrar el recurso. `with` automatiza ese ciclo de vida.

6. Context manager: adquirir, usar, liberar

```
with open(...) as file:
    use(file)
```

El patron conceptual es general:

```
ACQUIRE RESOURCE
       |
       v
USE RESOURCE
       |
       v
RELEASE RESOURCE
```

No solo existe para archivos. En software profesional se usa la misma idea para locks, conexiones, recursos temporales y otros objetos que deben cerrarse/liberarse correctamente.

7. El puntero/posicion del archivo

Cuando leemos un archivo, Python mantiene una posicion actual. `readline()` consume una linea y deja la posicion al comienzo de la siguiente.

```
header = file.readline()
# cursor is now after the header

for line in file:
    # starts from first data row
```

Por eso podemos consumir la cabecera y luego iterar sin volver a verla.

8. Por que CSV necesita una cabecera

Una fila como `100.0,20.0,0.0` no explica por si sola que significa cada numero.

```
h_m,v_m_s,t_s
```

La cabecera añade **semantica**: variable + unidad. En ingenieria esto es especialmente importante.

Un archivo sin unidades puede ser numericamente valido y fisicamente ambiguo.

9. Precision escrita vs precision interna

Un float Python se representa internamente en binario. Cuando hacemos:

```
f'{value:.6f}'
```

decidimos guardar solo seis decimales en texto. Eso es una decision de formato y potencialmente de precision.

Ejemplo conceptual:

```
internal value : 0.30000000000000004
written text   : 0.300000
loaded float   : 0.3
```

Por eso no exigimos igualdad binaria exacta despues del viaje; validamos dentro de una tolerancia.

10. Round-trip como prueba de integridad

Un round trip comprueba que nuestra pareja writer/reader esta de acuerdo.

```
A = original data
B = load(save(A))

desired property:
B approximately equals A
```

Esta es una idea muy potente: no probamos solo que 'se creo un archivo'; probamos que la informacion puede recuperarse correctamente.

11. Responsabilidad: por que file_io.py no es un metodo?

Podriamos escribir un metodo `earth_state.save_csv()`, pero en EX22 no es la mejor separacion conceptual.

VerticalState representa **fisica/estado**. CSV representa una **decision de almacenamiento**. Separar ambos evita acoplar la clase fisica a un formato concreto.

Pregunta	Lugar natural
Como evoluciona h,v,t?	VerticalState / physical model
Como calculo energia?	VerticalState / physical behavior
Como convierto una trayectoria a CSV?	file_io.py
Donde guardar el archivo?	main.py / application configuration

12. Separation of concerns

```
PHYSICS LAYER
vertical_state.py
      |
      v
APPLICATION / ORCHESTRATION
main.py
      |
      v
PERSISTENCE LAYER
file_io.py
      |
      v
FILESYSTEM
data/*.csv
```

Cada capa puede cambiar con menos impacto sobre las demas. Mañana podriamos reemplazar CSV por JSON o una base de datos sin reescribir la ecuacion de Euler.

13. Ruta de archivo vs contenido del archivo

Son conceptos distintos que a veces se mezclan:

Ruta	Contenido
Dice DONDE esta el recurso	Dice QUE informacion contiene
Path object / string	Texto/bytes
data/trajectory.csv	h_m,v_m_s,t_s ...

`Path` modela el primero. `open()` nos permite acceder al segundo.

14. Current Working Directory (CWD)

Una ruta relativa como `data/file.csv` se interpreta normalmente respecto a la **carpeta de trabajo actual del proceso**, no necesariamente respecto a la ubicacion del script.

```
terminal current directory
        |
        +-- relative paths are resolved from here
```

Eso explica muchos errores de 'el archivo existe pero Python no lo encuentra'. Anclar rutas a `Path(__file__).resolve().parent` hace que EX22 sea mas estable.

15. Modo w y destruccion de datos

`'w'` es util, pero tiene una consecuencia importante: si el archivo existe, su contenido se trunca antes de escribir.

```
old file contents
      |
open(..., 'w')
      |
      v
old contents removed
new contents written
```

Por eso no usamos 'w' para un historial acumulativo; para eso practicamos 'a'.

16. Modo a y logs

`'a'` coloca nuevas escrituras al final del archivo. Es adecuado para un log sencillo donde queremos conservar ejecuciones anteriores.

```
run 1 -> line 1
run 2 -> line 2 appended
run 3 -> line 3 appended
```

En sistemas reales los logs tienen herramientas mas avanzadas; aqui lo usamos para entender la semantica de append.

17. El formato es un contrato

Nuestro writer y loader comparten un contrato:

```
HEADER: h_m,v_m_s,t_s
ROW:    h,v,t
DELIMITER: comma
NUMERIC FORMAT: decimal text
```

Si mañana el writer cambia el orden a t,h,v y el loader sigue interpretando h,v,t, el programa puede leer sin error pero producir datos fisicamente incorrectos. Ese tipo de fallo es especialmente peligroso.

18. Validacion estructural y numerica

En EX22 comprobamos dos niveles:

```
STRUCTURE
trajectory.shape == loaded.shape

VALUES
np.allclose(trajectory, loaded)
```

La primera pregunta es 'recupere la misma estructura?'. La segunda es 'recupere los mismos valores dentro de la tolerancia?'.

19. Por que aun no usamos try/except

Existen muchos fallos posibles: archivo inexistente, permisos, fila corrupta, texto que no puede convertirse a float, etc. En EX22 dejamos que esos errores aparezcan para comprender primero el camino normal.

La siguiente clase de excepciones nos enseñara a controlar esos casos de manera explicita sin ocultar errores.

20. Conexion con ingenieria real

Problema	Persistencia tipica
GNSS/INS	measurement logs, trajectories, configs
Aircraft dynamics	time histories of states/forces
Propulsion	test-bench measurements
Astrodynamics	ephemerides, state histories
CFD/FEA	large scientific datasets

CSV es solo la primera capa. Mas adelante podriamos usar JSON para configuracion, NumPy binary, HDF5, NetCDF, databases o formatos especificos de cada dominio.

Resumen final - modelo mental

```
PYTHON OBJECT / ARRAY
        |
        | serialize / format
        v
TEXT REPRESENTATION
```

```
        |
        | encode + write
        v
PERSISTENT FILE
        |
        | read + decode
        v
TEXT REPRESENTATION
        |
        | parse + convert types
        v
PYTHON DATA AGAIN
        |
        | validate
        v
ROUND-TRIP CONFIDENCE
```

Idea central: File I/O no es simplemente 'guardar cosas'. Es diseñar una traduccion fiable entre la representacion interna de nuestro programa y una representacion persistente que otro proceso pueda recuperar.